NUMPY COMPLETE MASTERCLASS

NumPy Complete Masterclass

Hinglish Edition — Student Marks Dataset ke saath

12 Topics • 48 Functions • 192 Practice Questions • 1 Real Kaggle Dataset

Compiled by the NumPy Masterclass Team

Course Introduction

Is masterclass mein hum NumPy ko shuru se seekhenge — har function ka English definition, 4-4 practical examples, aur Hinglish mein asaan explanation ke saath. Poore course mein ek hi 'Student Marks Dataset' use hoga taaki continuity bani rahe: Aryan, Meera, Tanya, Isha aur Vikram — in 5 students ke Math, Science, English aur Computer ke marks. Jahan zaroorat hogi, wahan ek real Kaggle dataset (Students Performance in Exams) bhi use karenge, especially Topic 10 mein standard deviation aur variance samjhane ke liye.

Student Marks Dataset

Poore course mein neeche diya gaya dataset repeatedly use hoga. Isse ek baar run karke apne Python/Jupyter environment mein rakh lo:

```
import numpy as np
# Students: Aryan, Meera, Tanya, Isha, Vikram
# Columns: Math, Science, English, Computer
marks = np.array([
    [85, 90, 78, 92],   # Aryan
    [70, 65, 80, 75],   # Meera
    [95, 88, 92, 98],   # Tanya
    [60, 72, 55, 68],   # Isha
    [88, 85, 90, 86],   # Vikram
])
```

Kaggle Dataset Note (Topic 10 ke liye)

Real-world practice ke liye hum Kaggle ke mashhoor dataset 'Students Performance in Exams' (by Jakki Seshapanpu, kaggle.com/datasets/spscientist/students-performance-in-exams) ko refer karenge. Is dataset mein 1000 students ke math, reading aur writing scores hote hain — bilkul hamare Marks Dataset jaisa structure, bas bada scale.

Table of Contents

01 **Array Creation**

np.array(), np.zeros() / np.ones() / np.full(), np.arange() / np.linspace(), np.eye() / np.random.randint()

02 **Array Properties**

arr.shape, arr.ndim / arr.size, arr.dtype / arr.itemsize / arr.nbytes, arr.T (transpose)

03 **Indexing and Slicing**

arr[row, col] — single element access, arr[i] / arr[:, j] — row and column access, arr[start:stop] — slicing, Negative indexing & step slicing

04 **Math Operations**

Element-wise arithmetic (+, -, *, /, **), np.sqrt() / np.abs(), np.round() / np.floor() / np.ceil(), np.exp() / np.log()

05 **Aggregation Functions**

arr.sum(), arr.mean() / np.median(), arr.max() / arr.min() / arr.argmax() / arr.argmin(), arr.std() / arr.var()

06 **Reshaping**

arr.reshape(), arr.flatten() / arr.ravel(), arr.transpose() / arr.T, arr.resize()

07 **Masking and Filtering**

Boolean mask (arr > value), arr[mask] — filtering, np.where(), np.isin()

08 **Broadcasting**

Scalar broadcasting, Row-wise broadcasting (1D + 2D), Column-wise broadcasting (with keepdims), np.newaxis

09 **Sorting, Unique, Where, Clip, Dot**

np.sort(), np.unique(), np.clip(), np.dot()

10 **Spread of Data — std/var with Kaggle Dataset**

arr.std() — Standard Deviation, arr.var() — Variance, Loading a Kaggle CSV (pd.read_csv → .values), Coefficient of Variation (std / mean)

11 **Cumulative Operations — cumsum / cumprod**

arr.cumsum(), arr.cumprod(), cumsum(axis) vs cumprod(axis) behavior, Practical use: progress tracking with cumsum

12 **Multiple Aggregations Together**

getattr() based loop, Dict comprehension for stats, np.apply_along_axis(), pandas .agg() for tabular stats

TOPIC 01

Array Creation

Sabse pehle hume NumPy arrays banana aana chahiye — chahe list se ho, zeros/ones se ho, ya kisi range se. Neeche Aryan aur Meera ke examples ke through 4 sabse zaroori creation functions cover karte hain.

np.array()

ENGLISH DEFINITION

Creates a NumPy ndarray from a Python list or nested list of lists.

HINGLISH MEIN SAMJHO

Ye function sabse basic aur sabse zyada use hone wala function hai. Jab bhi humare paas already Python list ya nested list ke form mein data ho (jaise Aryan aur Meera ke marks), to hum np.array() ka use karke usse ek NumPy array mein convert karte hain. Nested list dene se automatically 2D array ban jaata hai, jahan outer list ki har entry ek row banti hai.

EXAMPLES

Example 1

```
>>> np.array([85, 90, 78, 92])
# Output: array([85, 90, 78, 92])  # Aryan's marks
```

Example 2

```
>>> np.array([1, 2, 3, 4, 5])
# Output: array([1, 2, 3, 4, 5])
```

Example 3

```
>>> np.array([[85,90],[70,65]])
# Output: array([[85, 90],
#                [70, 65]])  # Aryan & Meera
```

Example 4

```
>>> np.array([1,2,3], dtype=float)
# Output: array([1., 2., 3.])
```

PRACTICE QUESTIONS (HINGLISH)

1. Rohan aur Yash ke Math marks [88, 76] ka ek NumPy array banao.
2. Diya ke 4 subjects ke marks [90, 85, 88, 92] ko float type mein array banao.
3. Ankit, Mohit aur Mayank ke total marks [340, 355, 310] ka array banao.
4. Ek nested list se 3x2 ka array banao jisme Kushagra aur Arnav ke marks hon.

np.zeros() / np.ones() / np.full()

ENGLISH DEFINITION

np.zeros(shape) creates an array filled with 0s; np.ones(shape) fills with 1s; np.full(shape, value) fills with a custom value.

HINGLISH MEIN SAMJHO

Kabhi kabhi hume pehle se ek empty ya placeholder array chahiye hota hai jisme baad mein values fill karenge — jaise agar Tanya ka attendance record banana ho to pehle sab zeros se shuru karke baad mein update karenge. np.ones() bilkul same hai bas 1 se fill karta hai, aur np.full() apni marzi ki koi bhi value se fill karta hai, jaise grace marks ke liye saari values 5 se fill karna.

EXAMPLES

Example 1

```
>>> np.zeros((5, 4))
# Output: 5x4 zeros matrix — placeholder for 5 students, 4 subjects
```

Example 2

```
>>> np.ones(4)
# Output: array([1., 1., 1., 1.])
```

Example 3

```
>>> np.full((3,3), 7)
# Output: 3x3 matrix filled with 7
```

Example 4

```
>>> np.zeros_like(marks)
# Output: same shape as marks, sab zero
```

PRACTICE QUESTIONS (HINGLISH)

1. Isha ke liye 6 subjects wala ek zeros array banao.
2. 3 students (Ravi, Ankit, Mohit) ke liye ek ones matrix (3x4) banao.
3. Ek 4x4 matrix banao jisme har value 10 ho (grace marks ke liye).
4. marks array jaisa shape lekin sab zero wala array np.zeros_like() se banao.

np.arange() / np.linspace()

ENGLISH DEFINITION

np.arange(start, stop, step) generates evenly spaced values within a range; np.linspace(start, stop, num) generates a fixed number of evenly spaced points.

HINGLISH MEIN SAMJHO

np.arange() bilkul Python ke range() jaisa kaam karta hai lekin ek array return karta hai — jaise agar Vikram ke roll numbers 1 se 50 tak generate karne hon. np.linspace() thoda different hai: usme hum start aur end batate hain, aur kitne equally spaced points chahiye woh batate hain — ye tab useful hota hai jab exact count of points chahiye ho, jaise grading scale 0 se 100 tak 5 equal parts mein todna.

EXAMPLES

Example 1

```
>>> np.arange(1, 51)
# Output: array([1, 2, 3, ..., 50])  # roll numbers
```

Example 2

```
>>> np.arange(0, 20, 5)
# Output: array([0, 5, 10, 15])
```

Example 3

```
>>> np.linspace(0, 100, 5)
# Output: array([0., 25., 50., 75., 100.])
```

Example 4

```
>>> np.linspace(60, 100, 3)
# Output: array([60., 80., 100.])  # grade bands
```

PRACTICE QUESTIONS (HINGLISH)

1. 1 se 20 tak ke saare odd numbers np.arange() se banao.
2. 0 se 50 tak, step 10 ka array banao.
3. 0 se 1 tak 6 equally spaced points np.linspace() se banao.
4. Ananya ke liye 40 se 100 ke beech 7 grading checkpoints banao.

np.eye() / np.random.randint()

ENGLISH DEFINITION

np.eye(n) creates an n x n identity matrix; np.random.randint(low, high, size) generates random integers within a given range and shape.

HINGLISH MEIN SAMJHO

np.eye() ek identity matrix banata hai jisme diagonal pe 1 aur baaki jagah 0 hota hai — ye linear algebra aur machine learning mein bahut kaam aata hai. Dusri taraf np.random.randint() random test data generate karne ke liye use hota hai, jaise agar Abhinay ke liye fake practice marks generate karne hon bina manually type kiye.

EXAMPLES

Example 1

```
>>> np.eye(4)
# Output: 4x4 identity matrix
```

Example 2

```
>>> np.eye(3, dtype=int)
# Output: 3x3 identity matrix, integer type
```

Example 3

```
>>> np.random.randint(50, 100, (2,4))
# Output: 2x4 random marks between 50-99
```

Example 4

```
>>> np.random.randint(1, 7, 5)
# Output: array of 5 random dice rolls
```

PRACTICE QUESTIONS (HINGLISH)

1. Ek 5x5 identity matrix integer type mein banao.
2. Mayank ke liye 4 random marks (40 se 100 ke beech) generate karo.
3. 3 students ke liye 3x4 random marks matrix (50-100) banao.
4. np.eye(6) banao aur uska sum() nikalo — kya answer aayega, socho pehle.

TOPIC 02

Array Properties

Array banane ke baad sabse pehla kaam hota hai uske structure ko samajhna — shape kya hai, kitne dimensions hain, kitna memory le raha hai. Ye properties functions nahi hain, attributes hain, lekin utni hi important hain.

arr.shape

ENGLISH DEFINITION

A tuple attribute that returns the dimensions (rows, columns, ...) of an array.

HINGLISH MEIN SAMJHO

shape sabse pehla cheez hai jo kisi bhi array pe check karna chahiye. Ye batata hai array mein kitni rows aur kitne columns hain, tuple format mein. Jaise humare marks dataset mein 5 rows (5 students: Aryan, Meera, Tanya, Isha, Vikram) aur 4 columns (4 subjects) hain, to shape (5, 4) hoga.

EXAMPLES

Example 1

```
>>> marks.shape
# Output: (5, 4)
```

Example 2

```
>>> marks.T.shape
# Output: (4, 5)
```

Example 3

```
>>> np.array([1,2,3]).shape
# Output: (3,)
```

Example 4

```
>>> np.zeros((2,3,4)).shape
# Output: (2, 3, 4)
```

PRACTICE QUESTIONS (HINGLISH)

1. marks array ka shape print karke bolo kitne students aur kitne subjects hain.
2. Ek 3x5 array ka shape kya hoga? Code likh ke verify karo.
3. 1D array [10,20,30,40] ka shape kya hota hai?
4. marks.T ka shape marks ke shape se kaise different hai, explain karo.

arr.ndim / arr.size

ENGLISH DEFINITION

arr.ndim returns the number of dimensions (axes) of the array; arr.size returns the total count of elements.

HINGLISH MEIN SAMJHO

ndim batata hai array kitne dimension ka hai — 1D, 2D ya 3D. size batata hai total kitne elements array mein hain, chahe wo kisi bhi shape mein arranged ho. Marks dataset 2D hai (ndim=2) kyunki rows aur columns dono hain, aur size 20 hai kyunki 5 students x 4 subjects = 20 total marks entries hain.

EXAMPLES

Example 1

```
>>> marks.ndim
# Output: 2
```

Example 2

```
>>> marks.size
# Output: 20
```

Example 3

```
>>> np.array([1,2,3]).ndim
# Output: 1
```

Example 4

```
>>> np.zeros((2,3,4)).size
# Output: 24
```

PRACTICE QUESTIONS (HINGLISH)

1. marks.ndim aur marks.size dono print karo aur inka matlab batao.
2. Ek 3D array (2,2,2) banao aur uska ndim check karo.
3. Agar shape (10, 5) ho to size kya hoga? Calculate karo.
4. Rohan ke single subject ke marks ka array [88] banao aur uska ndim/size batao.

arr.dtype / arr.itemsize / arr.nbytes

ENGLISH DEFINITION

arr.dtype returns the data type of elements; arr.itemsize returns bytes per element; arr.nbytes returns total memory consumed by the array.

HINGLISH MEIN SAMJHO

Ye teeno properties memory aur data-type ke baare mein batati hain. dtype se pata chalta hai array mein integers hain ya floats. itemsize batata hai ek single element kitni memory (bytes) leta hai, aur nbytes total memory batata hai (itemsize x size). Ye tab kaam aata hai jab bade datasets ke saath kaam karte hain aur memory optimize karni hoti hai.

EXAMPLES

Example 1

```
>>> marks.dtype
# Output: dtype('int64')
```

Example 2

```
>>> marks.itemsize
# Output: 8
```

Example 3

```
>>> marks.nbytes
# Output: 160
```

Example 4

```
>>> np.array([1.5,2.5]).dtype
# Output: dtype('float64')
```

PRACTICE QUESTIONS (HINGLISH)

1. marks array ka dtype check karo — integer hai ya float?
2. Agar itemsize 8 bytes hai aur size 20 hai, to nbytes kya hoga?
3. Diya ke marks ko float32 dtype mein banao aur itemsize check karo.
4. int64 aur float64 ke itemsize mein kya difference hota hai?

arr.T (transpose)

ENGLISH DEFINITION

The .T attribute returns the transpose of the array — rows become columns and columns become rows.

HINGLISH MEIN SAMJHO

Transpose array ko 'flip' kar deta hai — jo pehle row thi wo ab column ban jaati hai. Marks dataset mein agar hum .T lagate hain, to ab har row ek subject ban jaata hai aur har column ek student — yani ab Math, Science, English, Computer rows ban jaate hain aur Aryan, Meera waghera columns.

EXAMPLES

Example 1

```
>>> marks.T
# Output: shape becomes (4, 5)
```

Example 2

```
>>> marks.T.shape
# Output: (4, 5)
```

Example 3

```
>>> marks.T[0]
# Output: array([85, 70, 95, 60, 88])  # Math column of all students
```

Example 4

```
>>> np.array([[1,2],[3,4]]).T
# Output: array([[1, 3],
#                [2, 4]])
```

PRACTICE QUESTIONS (HINGLISH)

1. marks.T karke dekho ab Math subject sabki values kaise dikhti hain.
2. Transpose ke baad shape (4,5) kyun ho jaata hai, explain karo.
3. Ek 3x2 array banao aur uska transpose nikaalo.
4. marks.T.T lagane par kya wapas original array milega? Verify karo.

TOPIC 03

Indexing and Slicing

Ab dekhte hain array ke andar se specific data kaise nikaale — ek student ka data, ek subject ka column, ya beech ke kuch students. Indexing aur slicing NumPy ki sabse zyada use hone wali skills hain.

arr[row, col] — single element access

ENGLISH DEFINITION

Accesses a single element at a specific row and column index using comma-separated indices.

HINGLISH MEIN SAMJHO

Agar humein bas ek specific value chahiye — jaise Aryan ka Science ka mark — to hum row aur column index comma se separate karke likhte hain. Yaad rahe indexing 0 se start hoti hai, to Aryan row 0 hai aur Science column 1 hai.

EXAMPLES

Example 1

```
>>> marks[0, 0]
# Output: 85  # Aryan's Math
```

Example 2

```
>>> marks[0, 1]
# Output: 90  # Aryan's Science
```

Example 3

```
>>> marks[2, 3]
# Output: 98  # Tanya's Computer
```

Example 4

```
>>> marks[4, 2]
# Output: 90  # Vikram's English
```

PRACTICE QUESTIONS (HINGLISH)

1. Meera ka English mark (row 1, col 2) nikalo.
2. Isha ka Computer mark nikalo.
3. Tanya ka Math mark nikalo.
4. Vikram ka Science mark nikalo aur print karo.

arr[i] / arr[:, j] — row and column access

ENGLISH DEFINITION

arr[i] selects an entire row at index i; arr[:, j] selects an entire column at index j using a full slice for rows.

HINGLISH MEIN SAMJHO

Agar puri row chahiye (ek student ke saare marks) to bas index likh dena kaafi hai. Agar puri column chahiye (jaise sab students ka Math ka mark) to hum ':' likhte hain rows ke liye (matlab 'sab rows') aur comma ke baad column index dete hain.

EXAMPLES

Example 1

```
>>> marks[0]
# Output: array([85, 90, 78, 92])  # Aryan's full row
```

Example 2

```
>>> marks[:, 0]
# Output: array([85, 70, 95, 60, 88])  # Math column, all students
```

Example 3

```
>>> marks[2]
# Output: array([95, 88, 92, 98])  # Tanya's row
```

Example 4

```
>>> marks[:, 3]
# Output: array([92, 75, 98, 68, 86])  # Computer column
```

PRACTICE QUESTIONS (HINGLISH)

1. Vikram ki puri row (saare 4 subjects ke marks) nikalo.
2. Sabhi students ke Science marks ka column nikalo.
3. Isha ki puri row nikalo aur usse print karo.
4. Sabhi students ke English marks ka column nikalo.

arr[start:stop] — slicing

ENGLISH DEFINITION

Extracts a sub-array between the start (inclusive) and stop (exclusive) index positions.

HINGLISH MEIN SAMJHO

Slicing se hum ek range of rows ya columns nikaal sakte hain, ek sath. Jaise agar hume sirf Meera se Isha tak ke students chahiye (index 1 se 3, kyunki stop exclusive hota hai), to hum marks[1:4] likhte hain. Ye same rule 2D slicing mein rows aur columns dono pe apply hoti hai.

EXAMPLES

Example 1

```
>>> marks[1:4]
# Output: Meera, Tanya, Isha ki rows
```

Example 2

```
>>> marks[:, 1:3]
# Output: Science + English columns for everyone
```

Example 3

```
>>> marks[1:3, 0:2]
# Output: Meera & Tanya's Math and Science only
```

Example 4

```
>>> marks[0:2]
# Output: Aryan & Meera's full rows
```

PRACTICE QUESTIONS (HINGLISH)

1. Aryan se Tanya tak (index 0 se 2) ke students ke marks nikalo.
2. Sirf Math aur Science columns (0 aur 1) sabke liye nikalo.
3. Tanya aur Isha ke sirf English aur Computer marks nikalo.
4. Last teen students (Tanya, Isha, Vikram) ke marks slicing se nikalo.

Negative indexing & step slicing

ENGLISH DEFINITION

Negative indices count from the end of the array (-1 is the last element); step slicing arr[::n] selects every nth element.

HINGLISH MEIN SAMJHO

Negative index bahut handy hota hai jab humein end se count karna ho — marks[-1] hamesha last student (Vikram) ki row dega, chahe array ka size kuch bhi ho. Step slicing (arr[::2]) har doosra element uthata hai — jaise alternate students pick karne ho to.

EXAMPLES

Example 1

```
>>> marks[-1]
# Output: array([88, 85, 90, 86])  # Vikram, last student
```

Example 2

```
>>> marks[-1, -1]
# Output: 86  # Vikram's last subject, Computer
```

Example 3

```
>>> marks[::2]
# Output: Aryan, Tanya, Vikram rows (index 0,2,4)
```

Example 4

```
>>> marks[::-1]
# Output: all rows reversed, Vikram first
```

PRACTICE QUESTIONS (HINGLISH)

1. Negative indexing se Isha (second-last student) ka data nikalo.
2. marks[-1,-1] ka use karke Vikram ka last subject ka mark nikalo.
3. Step slicing se alternate students (index 0, 2, 4) nikalo.
4. marks[::-1] laga kar poora array reverse order mein print karo.

TOPIC 04

Math Operations

NumPy ki asli power tab dikhti hai jab hum poore array pe ek saath mathematical operations apply karte hain — bina kisi loop ke. Isse hum bulk calculations, jaise sabke marks mein grace add karna, ekdum fast kar sakte hain.

Element-wise arithmetic (+, -, *, /, **)

ENGLISH DEFINITION

Applies arithmetic operators to every element of the array simultaneously (element-wise), without needing an explicit loop.

HINGLISH MEIN SAMJHO

Jab hum marks + 5 likhte hain, NumPy automatically har single element mein 5 add kar deta hai — chahe wo Aryan ka mark ho ya Vikram ka. Yehi 'vectorization' kehlata hai, aur ye loop se kaafi zyada fast hota hai. Isi tarah -, *, /, aur ** bhi element-wise apply hote hain.

EXAMPLES

Example 1

```
>>> marks + 5
# Output: 5 bonus marks add to everyone
```

Example 2

```
>>> marks * 2
# Output: all marks doubled
```

Example 3

```
>>> marks / 2
# Output: all marks halved (float result)
```

Example 4

```
>>> marks ** 2
# Output: all marks squared
```

PRACTICE QUESTIONS (HINGLISH)

1. Sabhi students ko 10 grace marks do (+ operator se).
2. Sabke marks ko 2 se multiply karke dekho kya hota hai.
3. 100 - marks karke 'kitne marks miss hue' nikalo.
4. Har mark ko square karke naya array banao.

np.sqrt() / np.abs()

ENGLISH DEFINITION

np.sqrt(arr) computes the element-wise square root; np.abs(arr) returns the element-wise absolute value.

HINGLISH MEIN SAMJHO

np.sqrt() har element ka square root nikalta hai — statistics mein standard deviation calculate karte waqt bahut use hota hai. np.abs() negative numbers ko positive bana deta hai, jaise agar hum 'marks lost' calculate karte waqt kisi student ka score negative aa jaaye to abs() use karke usse positive difference mein convert kar sakte hain.

EXAMPLES

Example 1

```
>>> np.sqrt(marks)
# Output: square root of every mark
```

Example 2

```
>>> np.sqrt(np.array([4,9,16]))
# Output: array([2., 3., 4.])
```

Example 3

```
>>> np.abs(np.array([-5, 3, -8]))
# Output: array([5, 3, 8])
```

Example 4

```
>>> np.abs(marks - 80)
# Output: absolute distance of every mark from 80
```

PRACTICE QUESTIONS (HINGLISH)

1. marks array ka square root nikalo np.sqrt() se.
2. Array [-10, 15, -20, 25] ka absolute value nikalo.
3. Har mark ka 80 se kitna absolute difference hai, nikalo.
4. np.sqrt([16, 25, 36, 49]) ka result kya aayega, predict karke verify karo.

np.round() / np.floor() / np.ceil()

ENGLISH DEFINITION

np.round(arr, decimals) rounds to the nearest given decimal; np.floor(arr) rounds down; np.ceil(arr) rounds up to the nearest integer.

HINGLISH MEIN SAMJHO

Ye teeno functions rounding ke different tareeke dete hain. np.round() normal rounding karta hai (jaise 77.6 → 78), np.floor() hamesha neeche wali value deta hai (77.6 → 77), aur np.ceil() hamesha upar wali value deta hai (77.6 → 78). Percentage ya average marks report karte waqt ye bahut kaam aate hain.

EXAMPLES

Example 1

```
>>> np.round(77.678, 1)
# Output: 77.7
```

Example 2

```
>>> np.floor(np.array([77.8, 65.2]))
# Output: array([77., 65.])
```

Example 3

```
>>> np.ceil(np.array([77.1, 65.9]))
# Output: array([78., 66.])
```

Example 4

```
>>> np.round(marks.mean(axis=1), 2)
# Output: rounded average per student
```

PRACTICE QUESTIONS (HINGLISH)

1. Har student ka average marks nikaalo aur 2 decimal tak round karo.
2. np.floor() se array [88.9, 76.1, 90.5] ko round-down karo.
3. np.ceil() se array [88.1, 76.9, 90.0] ko round-up karo.
4. Overall class average nikalo aur usse round karke 1 decimal tak dikhao.

np.exp() / np.log()

ENGLISH DEFINITION

np.exp(arr) computes e raised to the power of each element; np.log(arr) computes the natural logarithm of each element.

HINGLISH MEIN SAMJHO

Ye advanced math functions hain jo mostly machine learning (jaise softmax, log-loss) mein use hote hain. np.exp() exponential growth calculate karta hai, aur np.log() uska ulta operation hai — natural log nikalta hai. In dono ko samajhna future mein ML algorithms padhte waqt bahut kaam aayega.

EXAMPLES

Example 1

```
>>> np.exp(np.array([1,2,3]))
# Output: array([2.718, 7.389, 20.086])
```

Example 2

```
>>> np.log(np.array([1, np.e, np.e**2]))
# Output: array([0., 1., 2.])
```

Example 3

```
>>> np.exp(0)
# Output: 1.0
```

Example 4

```
>>> np.log(marks)
# Output: natural log of every mark
```

PRACTICE QUESTIONS (HINGLISH)

1. np.exp([0, 1, 2]) ka result nikalo aur samjho ye kaise grow karta hai.
2. marks array ka natural log nikalo np.log() se.
3. np.log(np.e) ka result kya aayega? Verify karo code se.
4. np.exp(np.log(marks)) lagane par original marks wapas milenge kya? Check karo.

TOPIC 05

Aggregation Functions

Aggregation functions poore array ko ek ya kuch values mein summarize karte hain — total, average, max, min waghera. axis parameter yahan sabse important concept hai: axis=0 matlab column-wise (subject-wise), axis=1 matlab row-wise (student-wise).

arr.sum()

ENGLISH DEFINITION

Returns the sum of array elements, optionally along a specified axis.

HINGLISH MEIN SAMJHO

sum() bina axis diye poore array ka ek grand total deta hai. axis=1 dene se har student ka total nikalta hai (row-wise sum), aur axis=0 dene se har subject ka total nikalta hai (column-wise sum). Ye difference samajhna bahut zaroori hai kyunki beginners yahi galti karte hain.

EXAMPLES

Example 1

```
>>> marks.sum()
# Output: 1552  # grand total of all marks
```

Example 2

```
>>> marks.sum(axis=1)
# Output: array([345, 290, 373, 255, 349])  # per student
```

Example 3

```
>>> marks.sum(axis=0)
# Output: array([398, 400, 395, 419])  # per subject
```

Example 4

```
>>> marks[0].sum()
# Output: 345  # Aryan's total
```

PRACTICE QUESTIONS (HINGLISH)

1. Har student ka total marks nikalo (axis=1 use karke).
2. Har subject ka total marks nikalo (axis=0 use karke).
3. Sirf Tanya ka total marks nikalo.
4. Poore class ka grand total kitna hai, calculate karo.

arr.mean() / np.median()

ENGLISH DEFINITION

arr.mean() computes the arithmetic average of elements; np.median() computes the middle value when data is sorted.

HINGLISH MEIN SAMJHO

mean() sabse common average hai — sum divide by count. median thoda alag hai, wo data ko sort karke beech ki value uthata hai, isliye outliers se kam affected hota hai. Agar Isha ke marks baaki sabse bahut kam hon, to mean usko 'pull down' karega lekin median utna affect nahi hoga.

EXAMPLES

Example 1

```
>>> marks.mean()
# Output: 77.6  # overall average
```

Example 2

```
>>> marks.mean(axis=0)
# Output: array([79.6, 80., 79., 83.8])  # subject-wise average
```

Example 3

```
>>> np.median(marks)
# Output: 82.5
```

Example 4

```
>>> marks.mean(axis=1)
# Output: array([86.25, 72.5, 93.25, 63.75, 87.25])  # student-wise average
```

PRACTICE QUESTIONS (HINGLISH)

1. Har subject ka average marks nikalo.
2. Har student ka average marks nikalo.
3. Poore dataset ka median nikalo.
4. Sirf Computer subject ka average nikalo.

arr.max() / arr.min() / arr.argmax() / arr.argmin()

ENGLISH DEFINITION

max()/min() return the largest/smallest value; argmax()/argmin() return the index position of the largest/smallest value.

HINGLISH MEIN SAMJHO

max() aur min() value batate hain, lekin argmax() aur argmin() us value ki 'position' (index) batate hain — ye tab useful hota hai jab humein jaanna ho ki KAUN sa student ya subject topper hai, sirf value nahi. axis ke saath use karke per-subject ya per-student topper bhi nikaal sakte hain.

EXAMPLES

Example 1

```
>>> marks.max()
# Output: 98  # highest single mark
```

Example 2

```
>>> marks.max(axis=0)
# Output: array([95, 90, 92, 98])  # topper mark per subject
```

Example 3

```
>>> marks.sum(axis=1).argmax()
# Output: 2  # Tanya is overall topper
```

Example 4

```
>>> marks.argmax(axis=0)
# Output: array([2, 0, 2, 2])  # topper index per subject
```

PRACTICE QUESTIONS (HINGLISH)

1. Poore dataset ka highest mark nikalo.
2. Har subject ka topper mark nikalo (axis=0).
3. Overall topper student ka index nikalo (sum + argmax se).
4. Kaunsa student sabse kam total marks laaya, argmin se pata karo.

arr.std() / arr.var()

ENGLISH DEFINITION

arr.std() computes the standard deviation (spread of data); arr.var() computes the variance (average squared deviation from the mean).

HINGLISH MEIN SAMJHO

std() aur var() batate hain data kitna 'scattered' hai. Agar std kam hai to matlab sab students ke marks ek dusre ke close hain (consistent). Agar std zyada hai to matlab kuch students bahut acche aur kuch bahut weak hain — jaise agar Tanya aur Isha ke marks mein bahut gap hai to std high aayega.

EXAMPLES

Example 1

```
>>> marks.std()
# Output: ~12.7  # overall spread
```

Example 2

```
>>> marks.var()
# Output: ~161  # overall variance
```

Example 3

```
>>> marks.std(axis=0)
# Output: array([13.42, 9.78, 13.83, 11.43])  # per subject spread
```

Example 4

```
>>> marks.std(axis=1)
# Output: array([5.79, 5.59, 3.56, 6.61, 2.06])  # per student consistency
```

PRACTICE QUESTIONS (HINGLISH)

1. Poore dataset ka standard deviation nikalo.
2. Har subject ka variance nikalo aur bolo kaunsa subject sabse zyada scattered hai.
3. Kaunsa student sabse consistent hai (sabse kam std), pata karo.
4. std aur var mein mathematical relationship kya hai? (Hint: var = std squared)

TOPIC 06

Reshaping

Reshaping se hum array ka data same rakhte hue uska structure (shape) badal sakte hain — bahut useful hai jab data ko kisi specific format mein chahiye ho, jaise ML models ke input ke liye.

arr.reshape()

ENGLISH DEFINITION

Returns a new array with the same data but a different shape, without changing the underlying data.

HINGLISH MEIN SAMJHO

reshape() total elements same rakhta hai, bas unhe naye rows/columns mein arrange kar deta hai. Marks array (5,4) = 20 elements ko hum (4,5), (2,10), ya (10,2) mein reshape kar sakte hain kyunki har case mein total 20 hi rehta hai. -1 dene se NumPy khud calculate kar leta hai woh dimension.

EXAMPLES

Example 1

```
>>> marks.reshape(4, 5)
# Output: new shape (4, 5), same 20 values
```

Example 2

```
>>> marks.reshape(-1)
# Output: flattened to shape (20,)
```

Example 3

```
>>> marks.reshape(2, 10)
# Output: shape (2, 10)
```

Example 4

```
>>> marks.reshape(2, 2, 5)
# Output: 3D array of shape (2, 2, 5)
```

PRACTICE QUESTIONS (HINGLISH)

1. marks array ko (4, 5) shape mein reshape karo.
2. marks ko -1 use karke 1D array mein convert karo.
3. marks ko (2, 2, 5) ke 3D shape mein reshape karo.
4. Kya marks ko (3, 7) mein reshape kar sakte hain? Try karke dekho kya error aata hai.

arr.flatten() / arr.ravel()

ENGLISH DEFINITION

Both convert a multi-dimensional array into a 1D array; flatten() always returns a copy, while ravel() returns a view when possible (faster, memory-efficient).

HINGLISH MEIN SAMJHO

Dono functions array ko 1D bana dete hain, lekin inme ek important difference hai: flatten() hamesha ek naya copy banata hai (original array safe rehta hai agar aap flattened wale mein change karo), jabki ravel() jahan tak ho sake original array ka hi 'view' deta hai — isliye ravel() thoda fast hota hai lekin usme change karne se original bhi change ho sakta hai.

EXAMPLES

Example 1

```
>>> marks.flatten()
# Output: 1D array, independent copy
```

Example 2

```
>>> marks.ravel()
# Output: 1D array, view (fast, shares memory)
```

Example 3

```
>>> marks.flatten().shape
# Output: (20,)
```

Example 4

```
>>> id(marks.ravel()) == id(marks)
# Output: False, but shares underlying data buffer
```

PRACTICE QUESTIONS (HINGLISH)

1. marks.flatten() karke 1D array banao aur uska shape check karo.
2. marks.ravel() use karke bhi 1D array banao.
3. flatten() aur ravel() mein main difference kya hai, apne words mein likho.
4. flatten() se milne wale array mein change karne par original marks change hota hai kya? Test karo.

arr.transpose() / arr.T

ENGLISH DEFINITION

Reverses or permutes the axes of an array — for a 2D array, rows and columns are swapped.

HINGLISH MEIN SAMJHO

transpose() aur .T dono same kaam karte hain 2D arrays ke liye — rows ko columns bana dete hain. Ye tab bahut kaam aata hai jab data ka orientation badalna ho, jaise agar humein subjects ko rows mein aur students ko columns mein dekhna ho.

EXAMPLES

Example 1

```
>>> marks.transpose()
# Output: same as marks.T, shape (4, 5)
```

Example 2

```
>>> marks.T
# Output: shape (4, 5)
```

Example 3

```
>>> marks.T[0]
# Output: array([85,70,95,60,88])  # Math for all students
```

Example 4

```
>>> np.array([[1,2,3]]).T
# Output: array([[1],[2],[3]])
```

PRACTICE QUESTIONS (HINGLISH)

1. marks.transpose() use karke array ko flip karo.
2. Transpose karne ke baad Science subject ki saari values kaise access karoge?
3. Ek 1x3 array ko transpose karke 3x1 banao.
4. marks.T aur marks.transpose() ka output same hota hai kya? Verify karo.

arr.resize()

ENGLISH DEFINITION

Changes the shape of an array in-place, modifying the original array's memory directly (unlike reshape which returns a new array).

HINGLISH MEIN SAMJHO

resize() aur reshape() mein sabse bada difference ye hai ki resize() original array ko hi directly modify kar deta hai (in-place), jabki reshape() ek naya array return karta hai bina original ko touch kiye. Isliye resize() use karte waqt savdhaan rehna chahiye kyunki original data permanently badal jaata hai.

EXAMPLES

Example 1

```
>>> a = marks.copy(); a.resize(4, 5)
# Output: a is now permanently (4, 5) shaped
```

Example 2

```
>>> a = np.array([1,2,3,4]); a.resize(2,2)
# Output: a becomes [[1,2],[3,4]] in-place
```

Example 3

```
>>> a.resize(2, 3)
# Output: pads extra spots with 0 if size increases
```

Example 4

```
>>> marks.copy().resize(20)
# Output: copy resized to 1D, original marks untouched
```

PRACTICE QUESTIONS (HINGLISH)

1. marks ka ek copy banao aur usse resize(4,5) se reshape karo.
2. resize() aur reshape() mein kya difference hai, explain karo.
3. Ek chhota array resize() se bada karo — extra jagah mein kya value aati hai?
4. Kya resize() original array ko change karta hai ya naya banata hai? Test karke batao.

TOPIC 07

Masking and Filtering

Boolean masking NumPy ka sabse powerful feature hai — isse hum conditions ke basis pe data filter kar sakte hain, jaise 'sirf pass hone wale marks dikhao' ya 'sirf 90 se upar wale marks nikaalo'.

Boolean mask (arr > value)

ENGLISH DEFINITION

Applying a comparison operator to an array returns a boolean array of the same shape, marking True where the condition holds.

HINGLISH MEIN SAMJHO

Jab hum marks > 80 likhte hain, NumPy har element ko check karta hai aur True/False ka ek same-shape array return karta hai. Ye 'mask' phir humein filtering ke liye use hota hai. Isse hum bina loop likhe pura dataset condition ke basis pe scan kar sakte hain.

EXAMPLES

Example 1

```
>>> marks > 80
# Output: boolean array, True where mark > 80
```

Example 2

```
>>> marks < 60
# Output: boolean array marking failing marks
```

Example 3

```
>>> marks == 90
# Output: True only where a mark is exactly 90
```

Example 4

```
>>> (marks > 80).sum()
# Output: count of marks greater than 80
```

PRACTICE QUESTIONS (HINGLISH)

1. marks > 90 ka boolean mask banao aur print karo.
2. Kitne marks 60 se kam hain, boolean sum se count karo.
3. marks == 85 ka mask banao — kaha kaha True hai?
4. Boolean mask ka use karke bolo total kitne 'A grade' (>=90) marks hain.

arr[mask] — filtering

ENGLISH DEFINITION

Using a boolean array as an index selects only the elements where the mask is True, returning a flattened 1D result.

HINGLISH MEIN SAMJHO

Ek baar mask ban jaaye, uss mask ko seedha array ke andar index ki tarah use kar sakte hain — jaise marks[marks > 80] sirf wo values return karega jo condition satisfy karti hain, baaki sab discard ho jaayenge. Result hamesha 1D hota hai chahe original array 2D ho.

EXAMPLES

Example 1

```
>>> marks[marks > 80]
# Output: array of all marks above 80
```

Example 2

```
>>> marks[marks < 60]
# Output: all failing marks
```

Example 3

```
>>> marks[marks == 98]
# Output: array([98])
```

Example 4

```
>>> marks[(marks > 70) & (marks < 90)]
# Output: marks strictly between 70 and 90
```

PRACTICE QUESTIONS (HINGLISH)

1. Sirf wo marks nikalo jo 90 se zyada hain.
2. Sirf failing marks (60 se kam) nikalo.
3. 70 aur 85 ke beech ke marks filter karo.
4. Vikram ki row se sirf wo marks nikalo jo 85 se zyada hain.

np.where()

ENGLISH DEFINITION

np.where(condition, value_if_true, value_if_false) returns an array choosing between two values based on a condition, element-wise.

HINGLISH MEIN SAMJHO

np.where() ek conditional 'if-else' hai jo poore array pe ek saath kaam karta hai. Jaise agar humein Pass/Fail labels chahiye har mark ke liye, to np.where(marks >= 60, 'Pass', 'Fail') condition true hone par 'Pass' aur false hone par 'Fail' de dega, same shape ke array mein.

EXAMPLES

Example 1

```
>>> np.where(marks >= 60, 'Pass', 'Fail')
# Output: Pass/Fail label array
```

Example 2

```
>>> np.where(marks > 80, 'A', 'B')
# Output: grade labels
```

Example 3

```
>>> np.where(marks > 90, marks, 0)
# Output: keeps marks > 90, else 0
```

Example 4

```
>>> np.where(marks < 60)
# Output: returns (row_indices, col_indices) of failing marks
```

PRACTICE QUESTIONS (HINGLISH)

1. np.where() se Pass/Fail labels banao (60 cutoff).
2. np.where() se A/B grade labels banao (80 cutoff).
3. 90 se kam marks ko 0 kar do, baaki same rakho, np.where() se.
4. np.where(marks < 60) chalake failing marks ke indices nikalo.

np.isin()

ENGLISH DEFINITION

np.isin(arr, values) tests each element of arr for membership in the given list of values, returning a boolean array.

HINGLISH MEIN SAMJHO

isin() check karta hai ki array ke elements ek given list mein hain ya nahi — jaise agar humein pata karna ho kaunse marks exactly [85, 90, 98] list mein se hain. Ye multiple '==' conditions ko OR karne se zyada clean tareeka hai.

EXAMPLES

Example 1

```
>>> np.isin(marks, [85, 90, 98])
# Output: True where mark matches any listed value
```

Example 2

```
>>> marks[np.isin(marks, [85,90,98])]
# Output: array([85, 90, 98])
```

Example 3

```
>>> np.isin(np.array([1,2,3,4]), [2,4])
# Output: array([False, True, False, True])
```

Example 4

```
>>> np.isin(marks, [100])
# Output: all False, since no student scored 100
```

PRACTICE QUESTIONS (HINGLISH)

1. np.isin() se check karo kaunse marks [70, 85, 95] list mein hain.
2. isin() ka use karke exactly [88, 92] wale marks filter karo.
3. Check karo kya kisi student ne exactly 100 score kiya hai, isin() se.
4. isin() aur simple == operator mein kya difference hai?

TOPIC 08

Broadcasting

Broadcasting NumPy ka wo rule hai jisse different shapes ke arrays bhi aapas mein operate kar sakte hain, bina manually unko same shape mein laaye. Ye bahut powerful lekin thoda confusing bhi ho sakta hai shuru mein.

Scalar broadcasting

ENGLISH DEFINITION

When an operation combines an array with a single scalar value, the scalar is automatically 'stretched' to match the array's shape.

HINGLISH MEIN SAMJHO

Ye broadcasting ka sabse simple type hai — jab marks + 5 karte hain, ye '5' apne aap har element ke saath match ho jaata hai, jaise usse pehle hi (5,4) shape ka bana diya gaya ho. Humein manually iske liye koi loop ya reshape nahi karna padta.

EXAMPLES

Example 1

```
>>> marks + 5
# Output: 5 added to every element
```

Example 2

```
>>> marks * 2
# Output: every element doubled
```

Example 3

```
>>> marks - 100
# Output: every element minus 100
```

Example 4

```
>>> marks > 75
# Output: scalar 75 broadcast for comparison
```

PRACTICE QUESTIONS (HINGLISH)

1. marks mein 15 grace marks add karo scalar broadcasting se.
2. Har mark ko 3 se multiply karo.
3. Broadcasting explain karo apne words mein — jab scalar array ke saath operate hota hai.
4. marks - 50 karke dekho kaunse results negative aate hain.

Row-wise broadcasting (1D + 2D)

ENGLISH DEFINITION

A 1D array with length matching the number of columns can be broadcast across every row of a 2D array.

HINGLISH MEIN SAMJHO

Agar humare paas subject-wise bonus hai jaise [5, 10, 0, 3] (Math ko 5, Science ko 10, English ko 0, Computer ko 3 bonus), to marks + [5,10,0,3] ye bonus row ko har student ki row mein automatically add kar dega, kyunki iski length (4) columns ki count se match karti hai.

EXAMPLES

Example 1

```
>>> marks + np.array([5, 10, 0, 3])
# Output: different bonus per subject, for everyone
```

Example 2

```
>>> marks * np.array([1, 1, 1.1, 1])
# Output: English marks scaled up 10%
```

Example 3

```
>>> marks + np.array([0,0,0,5])
# Output: only Computer gets +5 bonus
```

Example 4

```
>>> marks - np.array([80,80,80,80])
# Output: distance of every mark from 80
```

PRACTICE QUESTIONS (HINGLISH)

1. Har subject ko different bonus do: Math +5, Science +10, English +0, Computer +3.
2. Sirf Computer subject mein 5 extra bonus do, baaki same rakho.
3. Row-wise broadcasting ka rule kya hai (shapes match kaise hote hain)?
4. Har subject ke marks ko 80 se subtract karo (distance from 80).

Column-wise broadcasting (with keepdims)

ENGLISH DEFINITION

A column vector (shape (n,1)) can be broadcast across every column of a 2D array with n rows.

HINGLISH MEIN SAMJHO

Agar humein per-student bonus dena hai (jaise Aryan ko +5, Meera ko +10, waghera), to bonus ko column shape (5,1) mein dena padega, taaki wo har row ke saath match ho sake. Isko achieve karne ke liye hum reshape(-1,1) ya keepdims=True ka use karte hain.

EXAMPLES

Example 1

```
>>> marks + np.array([[5],[10],[0],[3],[8]])
# Output: different bonus per student
```

Example 2

```
>>> marks - marks.mean(axis=1, keepdims=True)
# Output: each student's marks minus their own average
```

Example 3

```
>>> marks.mean(axis=1).reshape(-1,1)
# Output: row averages reshaped for broadcasting
```

Example 4

```
>>> marks / marks.sum(axis=1, keepdims=True)
# Output: each mark's proportion of student's total
```

PRACTICE QUESTIONS (HINGLISH)

1. Har student ko different bonus do: [5,10,0,3,8] (column shape mein).
2. keepdims=True use karke har student ka apna average subtract karo.
3. Har mark ko student ke total ka proportion (fraction) nikalo.
4. Column-wise aur row-wise broadcasting mein kya farak hai, example ke saath batao.

np.newaxis

ENGLISH DEFINITION

np.newaxis is used to insert a new axis (dimension) into an array, converting a 1D array into a 2D row or column vector.

HINGLISH MEIN SAMJHO

Kabhi kabhi broadcasting ke liye humein 1D array ko manually 2D banana padta hai — np.newaxis exactly yahi karta hai. arr[:, np.newaxis] array ko column vector bana deta hai, aur arr[np.newaxis, :] row vector bana deta hai. Ye broadcasting rules ko explicitly control karne ka tareeka hai.

EXAMPLES

Example 1

```
>>> np.array([10,20,30])[:, np.newaxis]
# Output: shape becomes (3, 1), column vector
```

Example 2

```
>>> np.array([10,20,30])[np.newaxis, :]
# Output: shape becomes (1, 3), row vector
```

Example 3

```
>>> marks.mean(axis=1)[:, np.newaxis]
# Output: row-average as column vector for broadcasting
```

Example 4

```
>>> np.arange(5)[:, np.newaxis] + np.arange(3)
# Output: creates a (5,3) grid via broadcasting
```

PRACTICE QUESTIONS (HINGLISH)

1. Array [10,20,30] ko newaxis se column vector (3,1) mein convert karo.
2. Same array ko row vector (1,3) mein convert karo.
3. marks.mean(axis=1) ko newaxis se column bana kar marks se subtract karo.
4. newaxis kyun zaroori hota hai broadcasting mein, ek example ke saath explain karo.

TOPIC 09

Sorting, Unique, Where, Clip, Dot

Ye topic thoda mix hai — data ko order karna, duplicates hatana, values ko ek range mein restrict karna, aur matrices ko multiply karna. Ye sab bahut common operations hain real data analysis mein.

np.sort()

ENGLISH DEFINITION

Returns a sorted copy of the array along the specified axis (default is the last axis).

HINGLISH MEIN SAMJHO

np.sort() array ko ascending order mein sort karta hai. 2D array pe by default har row apni apni jagah pe sort hoti hai (axis=-1 ya axis=1). Agar humein har student ke marks unke apne row ke andar sort karke dekhne hon (lowest to highest), to axis=1 use karenge.

EXAMPLES

Example 1

```
>>> np.sort(marks, axis=1)
# Output: each student's marks sorted ascending
```

Example 2

```
>>> np.sort(marks, axis=0)
# Output: each subject's marks sorted ascending across students
```

Example 3

```
>>> np.sort(marks[0])
# Output: array([78, 85, 90, 92])  # Aryan's marks sorted
```

Example 4

```
>>> np.sort(marks, axis=1)[:, -1]
# Output: highest mark of each student
```

PRACTICE QUESTIONS (HINGLISH)

1. Har student ke marks ko row-wise sort karo.
2. Har subject ke marks ko column-wise sort karo.
3. Aryan ke marks ko individually sort karo.
4. Sorted array se har student ka highest mark nikalo.

np.unique()

ENGLISH DEFINITION

Returns the sorted array of unique (distinct) elements, removing duplicates.

HINGLISH MEIN SAMJHO

np.unique() saari duplicate values hata deta hai aur ek sorted list of distinct values deta hai. Ye tab useful hai jab humein jaanna ho ki dataset mein kitne alag alag score values hain, ya duplicate entries check karni hon.

EXAMPLES

Example 1

```
>>> np.unique(marks)
# Output: sorted array of all distinct mark values
```

Example 2

```
>>> np.unique(marks).size
# Output: count of unique mark values
```

Example 3

```
>>> np.unique([85, 90, 85, 78, 90])
# Output: array([78, 85, 90])
```

Example 4

```
>>> np.unique(marks, return_counts=True)
# Output: unique values with their frequency counts
```

PRACTICE QUESTIONS (HINGLISH)

1. marks array mein kitne unique values hain, pata karo.
2. np.unique() se saare distinct marks list karo.
3. return_counts=True use karke pata karo kaunsa mark sabse zyada repeat hua.
4. Array [90,90,85,78,78,78] ke unique values aur unke counts nikalo.

np.clip()

ENGLISH DEFINITION

np.clip(arr, min, max) limits (clamps) all values in the array to lie within the given [min, max] range.

HINGLISH MEIN SAMJHO

clip() koi bhi value jo min se neeche hai usse min bana deta hai, aur jo max se upar hai usse max bana deta hai — beech ki values untouched rehti hain. Jaise agar hum marks ko 65-95 ke range mein 'cap' karna chahte hain (moderation policy), to np.clip(marks, 65, 95) use karenge.

EXAMPLES

Example 1

```
>>> np.clip(marks, 65, 95)
# Output: values below 65 become 65, above 95 become 95
```

Example 2

```
>>> np.clip(marks, 70, 90)
# Output: tighter clamp between 70 and 90
```

Example 3

```
>>> np.clip([50, 105, 78], 60, 100)
# Output: array([60, 100, 78])
```

Example 4

```
>>> np.clip(marks - 10, 0, 100)
# Output: reduce all marks by 10 but never below 0
```

PRACTICE QUESTIONS (HINGLISH)

1. marks ko 70 se 95 ke range mein clip karo.
2. Array [45, 110, 88, 30] ko 50-100 ke range mein clamp karo.
3. Sabke marks 10 kam karo lekin 0 se neeche mat jaane do, clip() se.
4. clip() aur boolean masking se filtering mein kya difference hai?

np.dot()

ENGLISH DEFINITION

Computes the dot product (matrix multiplication) of two arrays.

HINGLISH MEIN SAMJHO

np.dot() do arrays ka matrix multiplication karta hai — ye simple element-wise multiply se bilkul alag hai. Agar hum marks ko uske transpose se multiply karte hain (marks aur marks.T), to result ek 5x5 matrix banta hai jo batata hai har do students ke marks patterns kitne 'similar' hain (higher value = zyada similar performance).

EXAMPLES

Example 1

```
>>> np.dot(marks, marks.T)
# Output: 5x5 similarity matrix between students
```

Example 2

```
>>> np.dot(np.array([1,2,3]), np.array([4,5,6]))
# Output: 32  # 1*4+2*5+3*6
```

Example 3

```
>>> marks @ marks.T
# Output: same as np.dot(), using @ operator
```

Example 4

```
>>> np.dot(marks.T, marks)
# Output: 4x4 matrix relating subjects to each other
```

PRACTICE QUESTIONS (HINGLISH)

1. marks aur marks.T ka dot product nikalo, result shape batao.
2. @ operator use karke wahi dot product nikalo.
3. marks.T aur marks ka dot product nikalo — shape kitni hogi?
4. Do simple 1D arrays [2,3,4] aur [1,0,1] ka dot product manually calculate karke verify karo.

TOPIC 10

Spread of Data — std/var with Kaggle Dataset

Ab hum standard deviation aur variance ko deeply samjhenge, aur ek real Kaggle dataset (Students Performance in Exams) load karke dikhayenge ki real-world data pe ye concepts kaise apply hote hain.

arr.std() — Standard Deviation

ENGLISH DEFINITION

Computes the standard deviation, a measure of how spread out the values are from the mean.

HINGLISH MEIN SAMJHO

Standard deviation batata hai data average se kitna 'door' scattered hai. Low std matlab students ke marks ek dusre ke close hain (sab consistent), high std matlab kuch bahut acche aur kuch bahut weak — jaise mix ho gaya ho. Ananya aur Abhinay jaise students agar dono consistent scorers hain to unka group std kam hoga.

EXAMPLES

Example 1

```
>>> marks.std()
# Output: ~12.7  # overall spread
```

Example 2

```
>>> marks.std(axis=0)
# Output: per-subject spread
```

Example 3

```
>>> marks.std(axis=1)
# Output: per-student consistency
```

Example 4

```
>>> marks.std(axis=0).argmax()
# Output: index of subject with most variation
```

PRACTICE QUESTIONS (HINGLISH)

1. Poore marks dataset ka overall standard deviation nikalo.
2. Kaunsa subject sabse zyada 'scattered' hai (highest std), pata karo.
3. Kaunsa student sabse consistent hai (lowest std), pata karo.
4. Std interpret karo: agar std bahut high ho to class ke baare mein kya keh sakte hain?

arr.var() — Variance

ENGLISH DEFINITION

Computes the variance, defined as the average of the squared differences from the mean (std squared).

HINGLISH MEIN SAMJHO

Variance aur standard deviation aapas mein directly related hain: variance = std ka square. Variance thoda abstract hota hai (units bhi square ho jaate hain, jaise 'marks squared'), isliye interpretation ke liye zyada std use hota hai, lekin variance mathematically statistics formulas mein bahut use hota hai.

EXAMPLES

Example 1

```
>>> marks.var()
# Output: ~161  # overall variance
```

Example 2

```
>>> marks.var(axis=0)
# Output: per-subject variance
```

Example 3

```
>>> np.sqrt(marks.var()) == marks.std()
# Output: True  # relationship verified
```

Example 4

```
>>> marks.var(axis=1)
# Output: per-student variance
```

PRACTICE QUESTIONS (HINGLISH)

1. Har subject ka variance nikalo.
2. Verify karo ki np.sqrt(variance) standard deviation ke barabar aata hai.
3. Har student ka variance nikal ke bolo kaun sabse zyada inconsistent hai.
4. Variance aur std mein practical use ka difference kya hai?

Loading a Kaggle CSV (pd.read_csv → .values)

ENGLISH DEFINITION

Uses pandas to read a real-world CSV dataset and converts numeric columns into a NumPy array via .values or select_dtypes().values.

HINGLISH MEIN SAMJHO

Real projects mein data hamesha CSV files mein aata hai, arrays mein nahi. Hum pandas se CSV load karte hain, phir sirf numeric columns nikaal kar unhe NumPy array mein convert karte hain. Isse hum real datasets, jaise Kaggle ke 'Students Performance in Exams' (1000 real students ke scores), pe bhi wahi std/var techniques apply kar sakte hain jo humne marks dataset pe seekhi.

EXAMPLES

Example 1

```
>>> import pandas as pd; df = pd.read_csv('StudentsPerformance.csv')
# Output: loads the Kaggle dataset into a DataFrame
```

Example 2

```
>>> arr = df.select_dtypes(include='number').values
# Output: converts numeric columns to a NumPy array
```

Example 3

```
>>> arr.std(axis=0)
# Output: std for math score, reading score, writing score columns
```

Example 4

```
>>> arr.mean(axis=0)
# Output: average of each numeric column across 1000 students
```

PRACTICE QUESTIONS (HINGLISH)

1. Kaggle 'Students Performance in Exams' dataset ko pandas se load karne ka code likho.
2. select_dtypes(include='number').values ka use kyun karte hain, explain karo.
3. Loaded array pe axis=0 std nikaalne se kya milega?
4. Apne khud ke marks dataset ko bhi ek CSV maan kar wahi workflow apply karo (conceptually).

Coefficient of Variation (std / mean)

ENGLISH DEFINITION

A normalized measure of spread, computed as standard deviation divided by mean, useful for comparing variability across datasets with different scales.

HINGLISH MEIN SAMJHO

Coefficient of variation (CV) tab useful hota hai jab hum do alag scales wale datasets ki spread compare karna chahte hon — jaise Marks dataset (0-100 scale) ko Kaggle ke reading score (jo shayad different range mein ho) se compare karna. CV ek unit-less ratio deta hai jisse fair comparison ho paata hai.

EXAMPLES

Example 1

```
>>> marks.std() / marks.mean()
# Output: ~0.164  # relative spread of marks dataset
```

Example 2

```
>>> arr.std(axis=0) / arr.mean(axis=0)
# Output: CV for each numeric column in Kaggle dataset
```

Example 3

```
>>> (marks.std(axis=0) / marks.mean(axis=0))
# Output: per-subject coefficient of variation
```

Example 4

```
>>> np.round(marks.std()/marks.mean(), 3)
# Output: rounded CV value
```

PRACTICE QUESTIONS (HINGLISH)

1. Marks dataset ka coefficient of variation nikalo.
2. Har subject ka alag-alag CV nikalo aur compare karo kaunsa subject relatively zyada scattered hai.
3. CV kyun 'unit-less' hota hai, explain karo.
4. CV ka use kis situation mein std se better hota hai?

TOPIC 11

Cumulative Operations — cumsum / cumprod

Cumulative functions running totals ya running products dete hain — har step pe pichle saare values ka combined result. Progress tracking, jaise test-by-test running score, ke liye bahut useful hote hain.

arr.cumsum()

ENGLISH DEFINITION

Returns the cumulative sum of array elements — each position holds the running total up to that point.

HINGLISH MEIN SAMJHO

cumsum() har index pe pichle saare elements ka total jama karta jaata hai. Jaise agar Aryan ke test-by-test marks [85, 90, 78, 92] hain, to cumsum uska 'running total after each test' dikhata hai: 85, 175, 253, 345. Ye progress tracking ke liye perfect hai.

EXAMPLES

Example 1

```
>>> np.array([85,90,78,92]).cumsum()
# Output: array([85, 175, 253, 345])
```

Example 2

```
>>> marks.cumsum(axis=1)
# Output: running total of marks for each student, test-by-test
```

Example 3

```
>>> marks.cumsum(axis=0)
# Output: running total of marks for each subject, student-by-student
```

Example 4

```
>>> marks[0].cumsum()
# Output: Aryan's running total across subjects
```

PRACTICE QUESTIONS (HINGLISH)

1. Aryan ke marks ka cumsum nikalo aur samjho har step ka matlab.
2. Har student ka test-by-test running total nikalo (axis=1).
3. Har subject ka running total across students nikalo (axis=0).
4. cumsum ka last value poore row ke sum ke barabar hota hai kya? Verify karo.

arr.cumprod()

ENGLISH DEFINITION

Returns the cumulative product of array elements — each position holds the running product up to that point.

HINGLISH MEIN SAMJHO

cumprod() bilkul cumsum() jaisa hai lekin addition ki jagah multiplication karta hai. Classic example hai factorial calculate karna: 1 se n tak ke numbers ka cumprod lene se hume 1!, 2!, 3!,... n! sab ek saath mil jaate hain.

EXAMPLES

Example 1

```
>>> np.arange(1,6).cumprod()
# Output: array([1, 2, 6, 24, 120])  # factorials
```

Example 2

```
>>> np.array([2,3,4]).cumprod()
# Output: array([2, 6, 24])
```

Example 3

```
>>> np.array([1,1,1,1]).cumprod()
# Output: array([1,1,1,1])
```

Example 4

```
>>> np.array([0.5, 2, 3]).cumprod()
# Output: array([0.5, 1., 3.])
```

PRACTICE QUESTIONS (HINGLISH)

1. 1 se 6 tak ke numbers ka cumprod nikal ke factorials list karo.
2. Array [2,3,4] ka cumsum aur cumprod dono nikal ke compare karo.
3. cumprod mein agar koi element 0 ho to uske baad ke saare values kya ban jaayenge?
4. 5! (5 factorial) ko cumprod se calculate karo.

cumsum(axis) vs cumprod(axis) behavior

ENGLISH DEFINITION

The axis parameter controls the direction of accumulation: axis=1 accumulates across columns within each row, axis=0 accumulates down rows within each column.

HINGLISH MEIN SAMJHO

2D arrays mein axis parameter bahut important ho jaata hai. axis=1 dene se har row ke andar accumulation hoti hai (jaise ek student ke test-by-test running score), jabki axis=0 dene se har column ke andar accumulation hoti hai (jaise ek subject mein student-by-student running total). Direction samajhna key hai.

EXAMPLES

Example 1

```
>>> marks.cumsum(axis=1)
# Output: row-wise accumulation, per student
```

Example 2

```
>>> marks.cumsum(axis=0)
# Output: column-wise accumulation, per subject
```

Example 3

```
>>> marks.cumprod(axis=1)[0]
# Output: running product across Aryan's subjects
```

Example 4

```
>>> marks.cumsum(axis=1)[:, -1]
# Output: same as marks.sum(axis=1) — final running total
```

PRACTICE QUESTIONS (HINGLISH)

1. axis=1 aur axis=0 ke cumsum results mein kya difference dikhta hai, compare karo.
2. cumsum(axis=1)[:, -1] aur sum(axis=1) same hote hain kya? Verify karo.
3. Tanya ke marks ka cumprod(axis nahi, sirf uski row pe) nikalo.
4. Real life mein axis=1 wala cumsum kab use karoge, ek example do.

Practical use: progress tracking with cumsum

ENGLISH DEFINITION

cumsum applied to a sequence of scores or values over time produces a running progress curve, commonly used for tracking cumulative performance.

HINGLISH MEIN SAMJHO

Ek practical use-case dekhte hain: agar Yash apne 5 practice tests ke marks track kar raha hai [60, 70, 75, 85, 90], to cumsum se pata chalega uska cumulative score kaise grow ho raha hai test-by-test — ye trend dikhane ke liye graph banane mein bhi useful hota hai.

EXAMPLES

Example 1

```
>>> np.array([60,70,75,85,90]).cumsum()
# Output: array([60, 130, 205, 290, 380])  # Yash's progress
```

Example 2

```
>>> np.array([60,70,75,85,90]).cumsum() / np.arange(1,6)
# Output: running average after each test
```

Example 3

```
>>> marks.cumsum(axis=1)[2]
# Output: Tanya's cumulative marks test-by-test
```

Example 4

```
>>> np.diff(np.array([60,130,205,290,380]))
# Output: array([70,75,85,90])  # reverse of cumsum
```

PRACTICE QUESTIONS (HINGLISH)

1. Yash ke 5 practice test marks [60,70,75,85,90] ka cumsum nikalo.
2. Usi array ka running average nikalo (cumsum divide by test number).
3. Tanya ka cumulative progress uske 4 subjects ke across nikalo.
4. np.diff() se cumsum ko wapas original values mein convert karke verify karo.

TOPIC 12

Multiple Aggregations Together

Course ke end mein dekhte hain kaise multiple statistics (sum, mean, max, min, std) ko ek saath, efficiently, loops ya dict comprehensions ke through nikala jaata hai — ye real reporting aur dashboards banane mein bahut kaam aata hai.

getattr() based loop

ENGLISH DEFINITION

getattr(object, 'method_name')(args) dynamically calls a method by its name given as a string, allowing iteration over multiple statistic names in a loop.

HINGLISH MEIN SAMJHO

Agar humein sum, mean, max, min, std — sab ek saath print karne hon bina 5 alag lines likhe, to getattr() kaam aata hai. Ye function names ko string se dynamically call karne deta hai, to hum ek list ['sum','mean','max'] pe loop chala kar har stat print kar sakte hain.

EXAMPLES

Example 1

```
>>> getattr(marks, 'sum')(axis=0)
# Output: same as marks.sum(axis=0)
```

Example 2

```
>>> for f in ['sum','mean','max']: print(f, getattr(marks,f)(axis=0))
# Output: prints 3 stats in a loop
```

Example 3

```
>>> getattr(marks, 'std')()
# Output: same as marks.std()
```

Example 4

```
>>> [getattr(marks, f)(axis=0) for f in ['min','max']]
# Output: list of min and max arrays
```

PRACTICE QUESTIONS (HINGLISH)

1. Ek loop banao jo sum, mean, max, min, std — paanchon stats print kare (axis=0).
2. getattr() explain karo — ye kaise kaam karta hai apne words mein.
3. getattr(marks, 'sum')(axis=1) chalake dekho kya result aata hai.
4. Ek list comprehension se sirf max aur min dono nikalo getattr() se.

Dict comprehension for stats

ENGLISH DEFINITION

A concise way to build a dictionary in a single line, mapping each statistic name to its computed result using getattr().

HINGLISH MEIN SAMJHO

Dict comprehension se hum saare stats ko ek clean dictionary mein store kar sakte hain, jaise {'sum': ..., 'mean': ..., 'max': ...} — isse baad mein har stat ko naam se access karna aasan ho jaata hai, jaise ek JSON report banate waqt.

EXAMPLES

Example 1

```
>>> {f: getattr(marks, f)(axis=0) for f in ['sum','mean','max']}
# Output: dict with 3 subject-wise stats
```

Example 2

```
>>> stats = {f: getattr(marks, f)(axis=0) for f in ['sum','mean','max','min','std']}
# Output: 5-stat summary dict
```

Example 3

```
>>> stats['mean']
# Output: accesses the mean array from the dict
```

Example 4

```
>>> {s.title(): getattr(marks,s)() for s in ['sum','mean']}
# Output: overall (not axis-wise) stats dict
```

PRACTICE QUESTIONS (HINGLISH)

1. Ek dict comprehension banao jisme sum, mean, max, min, std sab ho (axis=0).
2. Us dictionary se sirf 'mean' key ka value print karo.
3. Overall (axis nahi diya) sum aur mean ka dict banao.
4. Dict comprehension ka use karne ka fayda normal loop se kya hai?

np.apply_along_axis()

ENGLISH DEFINITION

Applies a custom function along a specified axis of an array, useful for calculations not directly available as built-in NumPy functions.

HINGLISH MEIN SAMJHO

Kabhi kabhi humein koi custom calculation chahiye hoti hai jo directly NumPy mein built-in nahi hai — jaise har student ka 'range' (max - min marks). apply_along_axis humein apna khud ka lambda function define karke usse har row (ya column) pe apply karne deta hai.

EXAMPLES

Example 1

```
>>> np.apply_along_axis(lambda x: x.max()-x.min(), 1, marks)
# Output: range (max-min) per student
```

Example 2

```
>>> np.apply_along_axis(lambda x: x.max()-x.min(), 0, marks)
# Output: range per subject
```

Example 3

```
>>> np.apply_along_axis(np.median, 1, marks)
# Output: median per student, same as np.median(marks,axis=1)
```

Example 4

```
>>> np.apply_along_axis(lambda x: (x>80).sum(), 1, marks)
# Output: count of marks above 80, per student
```

PRACTICE QUESTIONS (HINGLISH)

1. Har student ka range (max-min) apply_along_axis se nikalo.
2. Har subject ka range apply_along_axis se nikalo.
3. Ek custom function likho jo count kare kitne marks 85 se zyada hain, har student ke liye.
4. apply_along_axis kab use karna chahiye jab built-in function available na ho?

pandas .agg() for tabular stats

ENGLISH DEFINITION

The pandas DataFrame.agg() method applies one or more aggregation functions across columns, returning a neatly formatted summary table.

HINGLISH MEIN SAMJHO

Jab humein ek clean, readable table format mein multiple stats chahiye hon (jaise report banate waqt), to pandas ka .agg() bahut helpful hota hai. Ye numpy array ko pehle DataFrame mein convert karke, ek saath sum, mean, max jaise saare stats ko ek formatted table mein de deta hai.

EXAMPLES

Example 1

```
>>> pd.DataFrame(marks).agg(['sum','mean','max'])
# Output: table with sum, mean, max as rows
```

Example 2

```
>>> pd.DataFrame(marks, columns=['Math','Sci','Eng','Comp']).agg(['mean','std'])
# Output: labeled summary table
```

Example 3

```
>>> df.agg(['min','max','median'])
# Output: 3-row summary table
```

Example 4

```
>>> pd.DataFrame(marks).describe()
# Output: full statistical summary (count, mean, std, min, quartiles, max)
```

PRACTICE QUESTIONS (HINGLISH)

1. marks ko DataFrame banake sum, mean, max ka agg() table banao.
2. Column names (Math, Science, English, Computer) ke saath labeled table banao.
3. .agg() aur .describe() mein kya difference hai?
4. Ek table banao jisme sirf min, max, aur median ho.